Säkerhet i AI-system (DV2607)

Bilal Abdul
abac23@student.bth.se
Subhi Alloush
suao23@student.bth.se

Part 1: Central Adversarial Input Attacker

Del 1.1) Beskrivning av adversarial input attacker

Attack

Vi har valt att fokusera på en Evasion-attack, vilket är en av de vanligaste formen av adversarial input attack. En sådan attack sker i klassificeringssteget efter att maskininlärningsmodellen redan är färdigtränad. Grund iden är att en angripare manipulerar indatan till exempel genom att lägga till/ändra en pixel på en bild så att förändringen inte går att upptäcka med det mänskliga ögat, men ändå får modellen att fatta ett helt felaktigt beslut och klassificera bilden som en annan klass.

En välkänd grundmetod för detta är FGSM (Fast Gradient Sign Method), som är en så kallad white-box-attack där angriparen utnyttjar modellens gradienter. FGSM beräknar gradienten av modellens förlustfunktion med avseende på bildens indatapixlar. Genom att titta på tecknet (plus eller minus) för gradienten i varje pixel vet angriparen exakt åt vilket håll pixelvärdet ska ändras med ett litet steg för att maximera felet (vid en oriktad attack) eller för att styra modellen mot en specifik målklass (vid en riktad attack). En ännu starkare vidareutveckling av FGSM är PGD (Projected Gradient Descent), som kan beskrivas som en iterativ flerstegs-FGSM. Istället för att ta ett enda steg tar PGD flera små steg i gradientens riktning och projicerar efter varje steg tillbaka bilden inom en tillåten omgivning runt originalbilden, så att bruset förblir helt osynligt för ögat samtidigt som modellen övertygas med mycket hög säkerhet.

Skydd

För att skydda maskininlärningsmodeller mot adversarial input-attacker finns det huvudsakligen två kategorier av försvar: modellhärdning under träning samt förbehandling av indata.

Under själva träningsprocessen är en av de mest robusta metoderna Adversarial Training. Det innebär att man genererar fientliga exempel (med t.ex. FGSM eller

flerstegsattacken PGD) löpande under träningen och låter modellen tränas på både originalbilder och de manipulerade bilderna med rätt etiketter. Matematiskt formuleras detta som robust optimering (ett min-max-problem), där det inre problemet försöker maximera förlusten genom att skapa en så svår fientlig störning som möjligt, medan det yttre problemet uppdaterar modellens vikter för att minimera denna maximala förlust (Från vår Flervariabel kurs). Resultatet är att modellen jämnar ut sina beslutsgränser och slutar förlita sig på bräckliga, icke-robusta pixelmönster. När en angripare senare adderar brus ligger bilden fortfarande kvar på rätt sida om beslutsgränsen.

När man däremot använder en redan färdigtränad storskalig modell kan man skydda modellen genom Input Preprocessing, exempelvis Feature Squeezing och JPEG Compression. Dessa skydd fungerar som ett filter framför modellen som reducerar färgdjupet och filtrerar bort högfrekvent brus innan bilden klassificeras.

Del 1.2) Implementation av adversarial input attack

Så målet är att få ResNet50-modellen att klassificera bilden på koalan som en traktor utan att en människa ser någon förändring. Först kollar vi i klassifikationstabellen var koala och traktor ligger. Koala är i klass 105 och traktor är i klass 866. När vi kollar i ImageNets klasslista ser vi att koala ligger på plats 105 och traktor ligger på plats 866.

```
class_lookup = {105: 'koala', 866: 'tractor'}.
```

Som nämnt i del 1.1 ska vi implementera en riktad attack med PGD-metoden (Projected Gradient Descent). Vi ska använda TensorFlow och Keras (för att ladda ResNet50 och hantera bilderna) i vår implementation. Samt NumPy och Matplotlib för beräkningar och visualisering. Och ART, vilket står för Adversarial Robustness Toolbox. Specifikt ska vi använda följande klasser:

1. KerasClassifier

2. ProjectedGradientDescent

3. Hjälpfunktionen to_categorical

Innan attacken har ResNet50-modellen klassificerat originalbilden som klass 105 med 99,99 % konfidens, samt 0,00 % för traktor. Efter våra 15 iterationer av den riktade PGD-attacken har vi lyckats manipulera modellen så att den klassificeras som traktor.

Del 1.3) Säkerhetsåtgärder

Vi väljer att bygga ett kombinerat förbehandlingsskydd (Input Preprocessing Defence) för att skydda vår ResNet50-modell. Det består av Feature Squeezing med bit depth 4, i kombination med JPEG Compression med quality 60 via ART.

Feature Squeezing minskar bildens färgdjup från standarden på 8 bitar per färgkanal ner till 4 bitar. Det gör att närliggande pixelvärden slås ihop och avrundas till fasta steg. Detta resulterar i att merparten av dessa små störningar kommer att avrundas bort

tillbaka till originalpixelns kvantiserade värde, samtidigt som de tydliga visuella dragen hos en koala bevaras.

JPEG Compression komprimerar därefter bilden med kvalitetsfaktor 60 %. Den använder alltså cosinustransform och tar bort högfrekventa variationer som det mänskliga ögat inte uppfattar. Eftersom gradientbaserat brus från både FGSM och PGD till stor del består av just högfrekventa mönster fungerar JPEG som ett effektivt lågpassfilter som städar bort resterande fientligt brus.

Motivering till varför just denna metod:

1. Vi valde det här skyddsalternativet framför andra eftersom även om attacken i vår del 1.1, Adversarial Training, är en stark metod när man tränar en egen modell från grunden, är den opraktisk i vårt fall eftersom vår modell är en enormt förtränad modell (på typ över 1,2 miljoner bilder i ImageNet). Vi har inte computing-kraft för att träna all data som finns i ImageNet-datasetet. Därför är Feature Squeezing och JPEG Compression bra val eftersom de inte kräver någon omträning alls och kan placeras direkt framför befintliga modeller.

2. Vi kunde använda Postprocessing-skyddsmetoden (såsom HighConfidence eller ClassLabels). Efterbehandlingsskyddet ändrar bara modellens utdata för att försvåra black-box-attacker som Boundary Attack. Men de hindrar inte ett redan skapat white-box PGD-exempel från att lura de inre lagren i nätverket. Vårt förbehandlingsskydd stoppar bruset redan innan det når det första lagret i ResNet50.

3. Kraftig bildutjämning då? Ett för kraftigt utjämningsfilter riskerar att göra bilden så suddig att modellen tappar träffsäkerhet även på vanliga, oattackerade bilder. Genom att istället kombinera måttlig Feature Squeezing och JPEG Compression får vi ett flerlagerat skydd som bevarar hög säkerhet på den rena koalabilden, men samtidigt raderar det fientliga PGD-bruset.

Del 1.4) Implementering av skydd (frivillig för betyg A eller B)

I Jupyter Notebook implementerade vi en ny klassificerare (defended_classifier) genom att koppla ihop vår ResNet50-modell med våra två förbehandlingsfilter från ART: FeatureSqueezing och JpegCompression. För FeatureSqueezing är clip_values (0, 255) och bit depth = 4. För JpegCompression är clip_values = (0, 255) och quality = 60. Varje gång en bild skickas till vår nya modell passerar den automatiskt genom båda dessa skyddssteg innan ResNet50 gör sin klassificering.

För att kunna se resultaten av våra skydd jämförde vi originalmodellen och den härdade modellen i tre scenarier:

1. På de rena originalbilderna (koala och traktor) för att kontrollera att skyddet inte försämrar normal prestanda.

2. På den manipulerade koalabilden från del 1.2.

3. Vid en helt ny, direkt PGD-attack med samma parametrar riktad direkt mot den härdade modellen.

Testfall	Prediktad klass	P(Koala, klass=105)	P(Tractor, klass = 866)
Orgintalmodel på ren Koalabild	105(koala)	99%	9%
Originalmodell på attackerad bild (PGD)	866(tractor)	0%	99%
Härdad modell på ren koala bild	105(Koala)	99%	0%
Härdad modell på attackerad bild (från 1.2)	105(koala)	94%	0%
Härdad modell vid ny direkt PGD-attack	105(koala)	20%	1%

Slutsats:

Som tabellen visar hade PGD-attacken fullständig effekt på den oskyddade originalmodellen, där bilden klassificerades som en traktor. När samma manipulerade bild skickas in i vår härdade modell raderas det fientliga bruset effektivt och modellen klassificerar den igen som en koala med hög säkerhetsmarginal. Även när vi utför en ny PGD-attack direkt mot den härdade modellen misslyckas attacken med att tvinga fram målklassen traktor, och klassen koala förblir den mest sannolika klassen. Detta bevisar att vår kombination av Feature Squeezing och JPEG Compression ger ett mycket starkt och praktiskt skydd.

—

Part 2: Federated learning scenario

Part 2.1)

I den här delen har vi simulerat federerad inlärning med ramverket Flower. Vi använde datasetet CIFAR-10 som vi delade upp jämnt (IID) mellan 5 klienter, alltså 10 000 träningsbilder per klient. Modellen är ett enkelt CNN (SimpleCNN) med två faltningslager och två fullt kopplade lager. Varje klient tränar en epok lokalt per runda med SGD (learning rate 0,01, momentum 0,9 och batch size 32) och sedan skickar den sina vikter till servern. Servern slår ihop vikterna med FedAvg, alltså ett vanligt medelvärde, och utvärderar den globala modellen på testmängden med 10 000 bilder efter varje runda. Vi körde 35 rundor och mätte accuracy, F1-score och Cohens Kappa.

Attacken vi valde är en label flipping attack, vilket är en typ av data poisoning. En angripande klient byter ut alla sina etiketter enligt regeln ny etikett = 9 minus gammal etikett. Det betyder att flygplan (klass 0) blir lastbil (klass 9), bil (klass 1) blir skepp (klass 8), hjort (klass 4) blir hund (klass 5) och så vidare. Själva bilderna ändras inte alls så det går inte att se något konstigt på bilddatan, men modellen som tränas lokalt hos angriparen lär sig fel koppling mellan bild och klass. Servern ser aldrig klienternas data utan bara vikterna som skickas in, så den märker inte att en klient tränar på felaktiga etiketter.

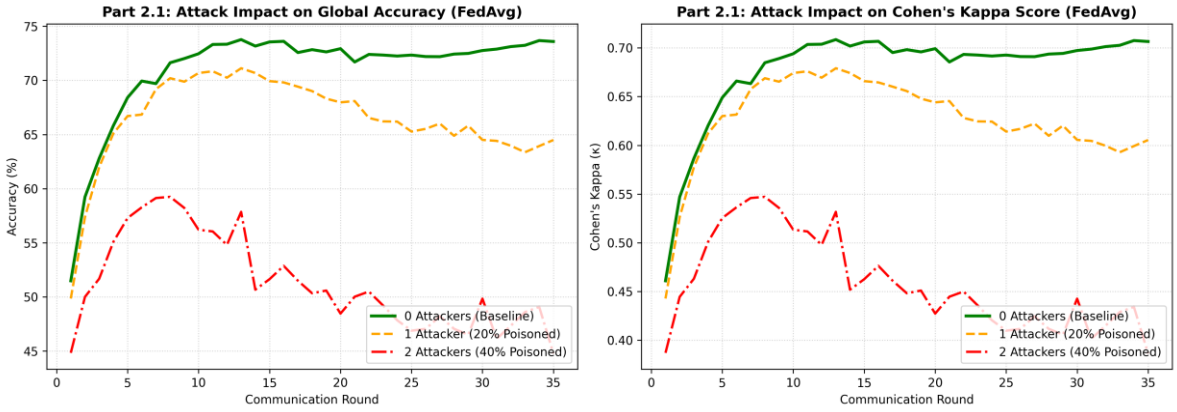
Vi testade tre olika fall över 35 rundor: 0 angripare (baslinje), 1 angripare (20 % av klienterna) och 2 angripare (40 % av klienterna).

1. Utan angripare (0 angripare): Modellen nådde 73,59 % accuracy, F1-score på 0,7360 och Cohens Kappa på 0,7066.

2. Med 1 angripare (20 % förgiftade klienter): Träffsäkerheten sjönk till 64,49 % accuracy, F1-score blev 0,6456 och Kappa sjönk till 0,6054. Det är ett tapp på drygt 9 procentenheter.

3. Med 2 angripare (40 % förgiftade klienter): Här blev det mycket värre och modellen landade på bara 44,76 % accuracy, F1-score 0,4472 och Kappa 0,3862.

Som man ser i Figur 1 nedan stiger alla tre kurvorna i början, men med en angripare börjar kurvan sakta gå nedåt efter ungefär runda 13. Med två angripare toppar den redan runt 59 % och sedan hoppar den upp och ner och faller ner under 45 %. Anledningen är att FedAvg tar ett vanligt medelvärde av alla inskickade vikter och därför får varje angripare lika stor vikt som en ärlig klient. Med två angripare är 40 % av alla uppdateringar förgiftade och de drar modellen åt fel håll i varje runda, så de ärliga klienterna hinner inte kompensera.



Figur 1: Påverkan av label flipping-attacken på accuracy och Cohens Kappa över 35 rundor med FedAvg.

Part 2.2)

För att skydda modellen på serversidan valde vi att byta ut FedAvg mot FedMedian. Istället för att ta medelvärdet av alla klienters vikter sorterar servern värdena i storleksordning för varje enskild parameter och väljer medianen (coordinate-wise median). Eftersom vi har 5 klienter blir det alltid det tredje värdet i ordningen som väljs. Om en angripare skickar konstiga eller extrema vikter hamnar de längst ut i kanterna av den sorterade listan och kommer då inte med i den globala modellen. I teorin har medianen en brytpunkt (breakdown point) på 50 %, vilket betyder att den klarar av upp till 2 angripare av 5 klienter så länge de ärliga klienterna är i majoritet.

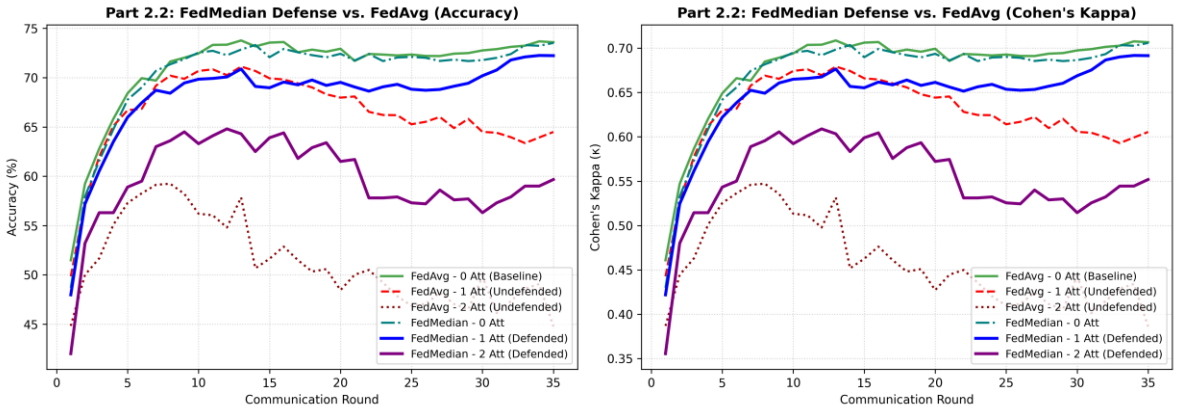
Vi valde FedMedian eftersom den är enkel att använda, den finns inbyggd i Flower och den kräver varken extra hyperparametrar eller någon separat valideringsdata på servern. Vi körde samma tre fall igen i 35 rundor:

1. Utan angripare (0 angripare): FedMedian fick 73,52 % accuracy och Kappa 0,7058, vilket är nästan exakt samma som FedAvg (73,59 %). Skyddet kostar alltså i princip ingenting i prestanda när alla klienter är ärliga.

2. Med 1 angripare (20 % förgiftade klienter): FedMedian nådde 72,23 % accuracy och Kappa 0,6914 jämfört med 64,49 % för FedAvg. Det är en förbättring med 7,74 procentenheter och nästan lika bra som helt utan attack.

3. Med 2 angripare (40 % förgiftade klienter): FedMedian nådde 59,67 % accuracy och Kappa 0,5519 jämfört med 44,76 % för FedAvg, vilket är nästan 15 procentenheter bättre.

Skyddet stoppar alltså en ensam angripare nästan helt, och vid två angripare förhindrar det att modellen rasar ihop även om den tappar en del jämfört med baslinjen. Att den tappar vid två angripare beror på att marginalen bara är 3 ärliga mot 2 angripare. Om de tre ärliga klienterna råkar ligga lite spritt på vissa parametrar kan en av angriparnas vikter hamna på tredje plats i sorteringen och väljas som median. En nackdel med FedMedian är också att den tar medianen för varje parameter separat, samt att den kan fungera sämre om datan är kraftigt Non-IID eftersom ärliga klienter då skiljer sig mer från varandra.



Figur 2: Jämförelse mellan oskyddad FedAvg och försvar med FedMedian vid 0, 1 och 2 angripare.

Del 2.3) Implemenation of protection (option if you aim for A and/or B grade)

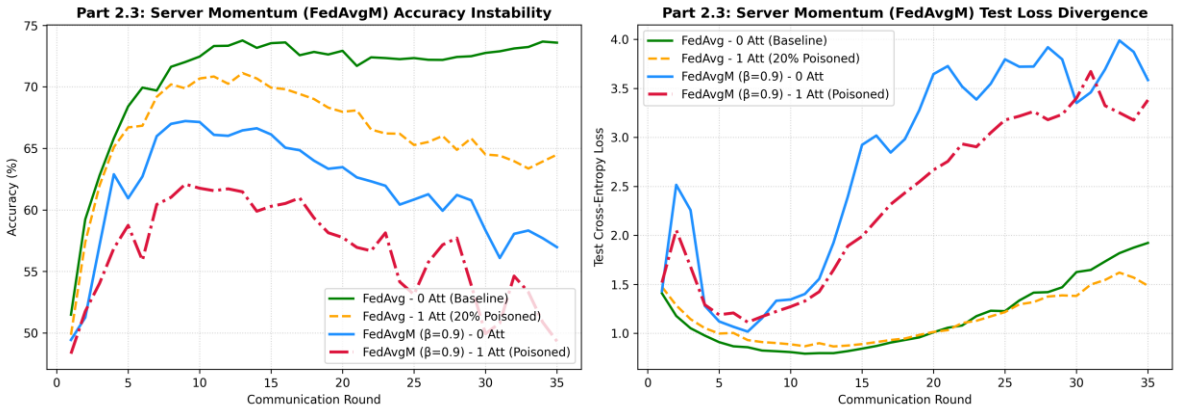
I den här delen testade vi FedAvgM, som är FedAvg med momentum på serversidan. Vi satte server_momentum till 0,9 och server_learning_rate till 1,0. Tanken med momentum är att servern sparar en hastighetsvektor från tidigare rundor och fortsätter i den riktningen, vilket brukar göra att träningen går snabbare och jämnar ut tillfälliga svängningar. Vi ville undersöka om den trögheten också kan fungera som ett skydd som dämpar förgiftade uppdateringar.

Vi körde FedAvgM i 35 rundor både utan angripare och med 1 angripare:

1. Utan angripare (0 angripare): FedAvgM steg snabbt i början och nådde som bäst 67,22 % i runda 9, men därefter blev träningen instabil och svängde kraftigt. Vid runda 35 låg modellen på 56,97 % accuracy, Kappa 0,5219 och en hög test loss på 3,5851.

2. Med 1 angripare (20 % förgiftade klienter): Modellen sjönk ner till 49,32 % accuracy, F1-score 0,4984, Kappa 0,4369 och loss 3,3812. Det är betydligt sämre än vanliga FedAvg med en angripare (64,49 %).

I vårt fall gjorde alltså FedAvgM resultatet sämre istället för bättre. Orsaken är hur momentum fungerar på servern. Servern uppdaterar sin hastighetsvektor som 0,9 gånger förra rundans vektor plus den nya medeluppdateringen. Eftersom servern fortfarande tar ett vanligt medelvärde filtreras inte angriparens vikter bort. Tvärtom sparas de förgiftade gradienterna kvar i hastighetsvektorn och byggs på runda efter runda. Dessutom gör ett högt momentum på 0,9 på servern tillsammans med momentum 0,9 lokalt i klienternas SGD att stegen blir för stora så att modellen skjuter över målet. Slutsatsen är att FedAvgM är en optimeringsmetod och inte ett försvar mot data poisoning.



Figur 3: FedAvgM (server momentum 0,9) jämfört med FedAvg för accuracy och test loss över 35 rundor.

Del 2.4) Implemenation of protection (option if you aim for A)

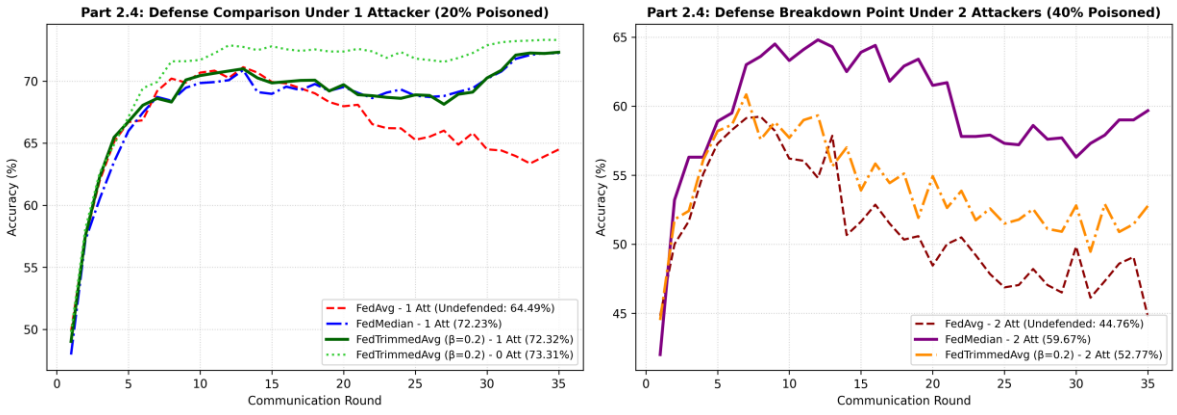
För betyg A implementerade vi ytterligare ett försvar, FedTrimmedAvg (coordinate-wise trimmed mean), och jämförde alla strategier mot varandra. För varje parameter sorterar servern värdena från alla 5 klienter och tar bort en andel beta av de lägsta och högsta värdena innan den räknar ut medelvärdet på de som är kvar i mitten. Vi satte beta = 0,2. Med 5 klienter betyder det att servern tar bort det lägsta och det högsta värdet (1 klient i varje ände) och beräknar medelvärdet av de 3 mittersta klienterna.

Resultaten för FedTrimmedAvg över 35 rundor blev:

1. Utan angripare (0 angripare): 73,31 % accuracy, F1-score 0,7329 och Kappa 0,7034. Precis som FedMedian ger den alltså nästan ingen förlust alls på ren data jämfört med FedAvg (73,59 %).

2. Med 1 angripare (20 % förgiftade klienter): 72,32 % accuracy, F1-score 0,7230 och Kappa 0,6924. Här presterar FedTrimmedAvg till och med lite bättre än FedMedian (72,23 %). Det beror på att när den ensamma angriparen har klippts bort i ena änden räknar FedTrimmedAvg medelvärdet av 3 ärliga klienter istället för att bara ta en enda klients värde som medianen gör.

3. Med 2 angripare (40 % förgiftade klienter): 52,77 % accuracy, F1-score 0,5277 och Kappa 0,4752. Det är klart bättre än oskyddad FedAvg (44,76 %), men sämre än FedMedian (59,67 %). Förklaringen är att med beta = 0,2 tas bara 1 klient bort i varje ände. När 2 angripare drar åt samma håll klipps bara den ena bort medan den andra angriparen kommer med i medelvärdet bland de 3 mittersta.



Figur 4: Jämförelse mellan FedTrimmedAvg, FedMedian och FedAvg vid 1 angripare (vänster) och 2 angripare (höger).

Sammanfattande tabell och slutsats för hela Del 2:

Tabellen nedan sammanfattar resultaten vid runda 35 för samtliga 11 simuleringar som vi har kört:

Strategi	Fördelning	Angripare	Andel förgiftade	Accuracy	F1-Score	Cohens Kappa	Test Loss
FedAvg (Baslinje)	IID	0	0 %	73,59 %	0,7360	0,7066	1,9223
FedAvg	IID	1	20 %	64,49 %	0,6456	0,6054	1,4851
FedAvg	IID	2	40 %	44,76 %	0,4472	0,3862	2,3883
FedMedian	IID	0	0 %	73,52 %	0,7351	0,7058	1,8365
FedMedian	IID	1	20 %	72,23 %	0,7220	0,6914	1,7770
FedMedian	IID	2	40 %	59,67 %	0,5958	0,5519	1,9546
FedAvgM (beta=0,9)	IID	0	0 %	56,97 %	0,5793	0,5219	3,5851
FedAvgM (beta=0,9)	IID	1	20 %	49,32 %	0,4984	0,4369	3,3812
FedTrimmedAvg (beta=0,2)	IID	0	0 %	73,31 %	0,7329	0,7034	1,9943
FedTrimmedAvg (beta=0,2)	IID	1	20 %	72,32 %	0,7230	0,6924	1,7818
FedTrimmedAvg (beta=0,2)	IID	2	40 %	52,77 %	0,5277	0,4752	2,0242

Sammanfattningsvis ser vi att vanlig FedAvg är väldigt sårbar mot label flipping och tappar nästan 29 procentenheter vid två angripare. FedAvgM fungerar inte som ett skydd eftersom momentum sparar de förgiftade gradienterna över tid. Både FedMedian och FedTrimmedAvg är däremot effektiva försvar. När det bara finns 1 angripare (20 %) är FedTrimmedAvg med beta = 0,2 allra bäst (72,32 %) eftersom den klipper bort angriparen och tar medelvärdet av tre ärliga klienter. Men när antalet angripare ökar till 2 (40 %) är FedMedian klart starkast (59,67 % mot 52,77 %) eftersom medianen klarar upp till 50 % angripare utan att man behöver ställa in någon trimningsparameter i förväg.